

Reflection: Parallelism and Concurrency in Python

Introduction

This assignment implements a thread-safe banking system in Python, consisting of a `BankAccount` class supporting concurrent deposits, withdrawals, and transfers, and a `TransactionSimulator` that stress tests the system under concurrent load. A banking system is a particularly instructive domain for this exercise, as the consequences of concurrency failures on an account balance are immediate and concrete. This document covers the key design decisions made during implementation, the concurrency strategy employed, the testing approach, and a critical evaluation of the implementation's limitations relative to a production banking system.

Design Decisions and Patterns

Lock over Semaphore

`BankAccount` uses `threading.Lock` rather than `threading.Semaphore` to protect the account balance. A semaphore permits concurrent accessors, which is appropriate for resource pools but unsuitable here. Only one thread should ever modify the balance at a time, making strict mutual exclusion the correct primitive (Quan, 2022, ch.14).

Encapsulation

The lock is a private instance attribute, invisible to callers. Thread safety is guaranteed by the class itself rather than depending on callers to manage locking explicitly. It is worth noting that Python's private convention is not enforced by the language. Nothing prevents code from bypassing the lock by directly manipulating `_balance`, as lock objects are not connected to the resources they protect (Quan, 2022, ch.14). In a production banking system this would represent a meaningful security concern.

Template Method Pattern

`User` subclasses `threading.Thread` and overrides `run()`, applying the Template Method pattern wherein a parent class defines an algorithm's skeleton and delegates specific behaviour to subclasses (Sarcar, 2023, ch.1). This was preferred over a plain target function because each simulated user has its own state, which benefits from encapsulation within the object.

SOLID Dependency Inversion Principle

`TransactionSimulator` receives its `BankAccount` instance as a constructor argument rather than creating one internally (Sarcar, 2023, ch.1). This decouples the simulator from the specific account it operates on and makes the system directly testable against accounts with known starting balances.

Lock Granularity in transfer()

transfer() manages its own locks rather than delegating to deposit() and withdraw(). Both of those methods acquire self._lock internally, so delegating would cause the thread to attempt acquiring a lock it already holds, resulting in reentrant deadlock. Managing locks directly at a finer granularity gives transfer() full control over acquisition order, which is central to the deadlock prevention strategy described in the following section. This requires accessing _balance directly on both account objects, a known and accepted tradeoff between correctness and strict encapsulation noted in the code comments (McDonald, 2023, ch.17).

Concurrency and Deadlock Prevention

Mutual Exclusion and Context Management

All methods that interact with the account's balance protect their critical sections using Python's with self._lock: construct (McDonald, 2023, ch.17). This guarantees lock release on exit even if an exception occurs, eliminating the class of bugs where an unreleased lock causes all waiting threads to block indefinitely. Critical sections are kept as small as possible, as holding a lock longer than necessary reduces concurrency benefits (Quan, 2022, ch.14).

Resource Ordering for Multi-Lock Transactions

The transfer() method is the only point where two locks must be held simultaneously. Without intervention, two concurrent transfers in opposite directions will each acquire one lock and wait indefinitely for the other, satisfying Coffman's circular wait condition (Quan, 2022, ch.12). The solution is resource ordering: locks are always acquired in ascending order of object identity using Python's id() function, regardless of transfer direction. This eliminates circular wait, an approach Quan (2022, ch.12) identifies as a primary deadlock prevention strategy.

Identity Verification for Self-Transfers

A self-transfer, where source and target are the same object, would cause an immediate deadlock: the thread acquires the lock via lock_a and blocks indefinitely attempting to re-acquire the same lock via lock_b. An object identity guard (if self is target_account:) tests for this before any lock interaction, failing fast with a ValueError.

Deterministic Stress Testing via Barriers

TransactionSimulator uses threading.Barrier to release all user threads simultaneously, maximizing lock contention and ensuring the locking mechanisms are exercised under genuine concurrent pressure rather than accidental sequential execution.

Testing and Validation

The test suite is divided into functional tests, which verify correctness in isolation, and concurrency tests, which validate thread safety under load. McDonald (2023, ch.20) describes unit tests as testing a

single unit of behaviour with a known input and an asserted output. The functional tests follow this principle:

- Deposit increases balance by the correct amount
- Withdrawal decreases balance by the correct amount
- Withdrawal raises `ValueError` when funds are insufficient
- Transfer deducts from source and credits target correctly
- Transfer raises `ValueError` when source funds are insufficient
- Transfer to the same account raises `ValueError` and leaves the balance unchanged

Concurrency tests validate behaviour that cannot be observed in single-threaded execution.

- The concurrent deposit test is deterministic: two threads each making five deposits of one hundred dollars against a zero balance must always produce one thousand dollars.
- Any deviation is the signature of a race condition (Quan, 2022, ch.14).
- The mixed transaction test runs one hundred simulated users against a single account and asserts the balance never goes negative, which would indicate two threads simultaneously bypassing the balance check before either had written back the decremented value.
- The deadlock test constructs two threads performing transfers in opposite directions simultaneously, joined with a five second timeout (McDonald, 2023, ch.17). A thread still alive after the timeout has deadlocked and the test fails.

Test output is provided in the appendix.

Limitations and Evaluation

Python's Global Interpreter Lock (GIL) ensures only one thread executes bytecode at a time, meaning this implementation achieves concurrency but not true parallelism (Quan, 2022, ch.15). For a banking system with brief, largely IO-bound operations, this is unlikely to be significant in practice. However, the GIL does not substitute for application-level locking. Threads still interleave and race conditions remain possible without explicit locks (Quan, 2022, ch.15). The two mechanisms serve different purposes and neither replaces the other.

Python's conventional rather than enforced encapsulation represents security risk. Nothing prevents code from directly manipulating `_balance`, bypassing the lock entirely and reintroducing the race conditions the implementation is designed to prevent. Refactoring to use `threading.RLock` throughout would allow `transfer()` to safely delegate to `deposit()` and `withdraw()`, restoring full encapsulation while eliminating the reentrant deadlock risk (McDonald, 2023, ch.17). This represents a clear path to improvement not taken here in the interest of keeping the locking strategy explicit.

This implementation demonstrates concurrency principles rather than production-grade banking capabilities. A real system would require persistent storage, transaction logging, rollback on failure, authentication, and audit trails. The simplicity is intentional, allowing the concurrency mechanisms to be examined without production grade noise.

References

McDonald, J. C. (2023) Dead Simple Python: Idiomatic Python for the Impatient Programmer. 1st edition. New York: No Starch Press.

Quan, Nguyen (2022) Advanced Python Programming. 2nd edition. Birmingham: Packt Publishing.

Sarcar, V. (2023) Java Design Patterns: A Hands-On Experience with Real-World Examples. 3rd edition. Berkeley, CA: Apress.

Appendix A: Unit Test Output

```
fchagnon@fchagnon-ThinkPad-T490:~$ python3 -m unittest -v test_bank.py
test_concurrent_bidirectional_transfers_do_not_deadlock (test_bank.TestBankAccount) ... ok
test_concurrent_deposits_produce_correct_final_balance (test_bank.TestBankAccount) ... ok
test_concurrent_mixed_transactions_never_produce_negative_balance (test_bank.TestBankAccount) ... ok
test_deposit_increases_balance_by_correct_amount (test_bank.TestBankAccount) ... ok
test_transfer_deducts_from_source_and_credits_target (test_bank.TestBankAccount) ... ok
test_transfer_raises_value_error_when_insufficient_funds (test_bank.TestBankAccount) ... ok
test_transfer_to_self_raises_value_error (test_bank.TestBankAccount)
Ensures transferring money to the exact same account instance ... ok
test_withdraw_decreases_balance_by_correct_amount (test_bank.TestBankAccount) ... ok
test_withdraw_raises_value_error_when_insufficient_funds (test_bank.TestBankAccount) ... ok
```

```
-----
Ran 9 tests in 0.028s
```

```
OK
```

Observations:

- All 9 tests passed in 0.029 seconds.
- The rapid completion of the deadlock test confirms that both transfer threads finished without deadlocking, well within the 5 second timeout threshold.